Evaluating the Accuracy of Ultrasonic Waves and Force For Determining Distance

Eduardo Ambriz Lazo[1], Lily Anfang[1], Nicholas Dass[1], and Quinn Miyamoto[1], *Team 37 Section 3, May 9th, 2025*

***Abstract*— This report details the design methodology and subsequent experiment of an autonomous underwater vehicle (AUV) equipped with a variety of sensors. The goal of this project, the final project for Harvey Mudd College's Experimental Engineering course, was to design an autonomous vehicle that could adequately determine the distance between itself and an object it was following. As such, the vehicle was outfitted with an ultrasonic distance sensor, a hall effect sensor, and force sensing resistor (FSR). This experiment would serve to analyze the efficiency of the ultrasonic distance sensor and FSR in measuring distance, acknowledging the flaws of each method and the associated cost, setup, and mechanical implementation. Ideally, the ultrasonic distance sensor would output distance data that could be integrated into a feedback loop to control motor PWM and distance from a measured object to maintain a constant following distance. However, due to technical difficulties experienced from rain, the project shifted to focusing on proving the novel application of object detection from a FSR. The vehicle was ultimately deployed at Baby Beach, Dana Point, CA. Incidentally, this report finds the implementation of Hooke's Law necessary for evaluating force using a FSR. Given the pre-specified distance during deployment, the FSR was relatively accurate, outputting a voltage range from 1-3 V corresponding to force readings of 0.36, 0.853, and 1.36 N. The anemometer (constructed with a hall effect sensor) was also able to collect variable wind speeds (from 2 m/s to 6.21 m/s) from manual rotation and ambient conditions.**

I. Introduction

A. Engineering Goal

This project explored the potential of using an ultrasonic distance sensor and force sensitive resistor (in a voltage divider) to determine the relative distance of an AUV. This would primarily target the efficiency of the sensors used in the circuit as well as a creative mechanical enhancement to convert force into a measurable distance. Relative results compared to ground truth would enable the evaluation of each sensor, with the wind results serving as an opportunity to explain discrepancies. The particular sensors utilized in this experiment employ various methods of determining distance or Δx. First off, Hooke's Law could be combined with a force sensing resistor and spring (with determined spring constant *k*). Secondly, ultrasonic wave packets, sent out in pulses, could be tracked and received to determine the distance from an object given the time of retrieval. The sensors utilized in the final deployment ensure these two concepts are properly explored and adapted to interact with the motherboard and Teensy interface provided in the Experimental Engineering course.

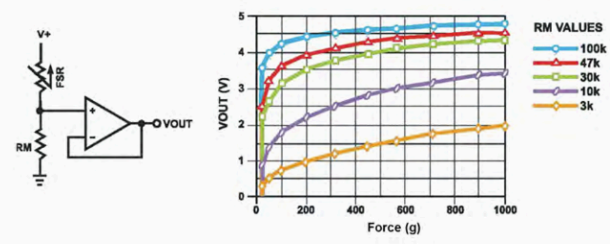
B. Sensor Selection & Relevant Concepts

In considering proper ways of measuring distance, the topic of Hooke's Law came up. Specifically, the idea of using a mechanical spring system and a sensor that could adequately measure force. FSRs are typically used for pressure measurements, as clarified in the data sheet, but the mechanical integration of such a sensor to compare against ultrasonic waves was appealing [1]. It was expected that as an applied force, taken from experiment, combined with a spring constant *k* could allow for the derivation of a distance Δx (1). This led to the decision to use an FSR, which is "a piezoresistive sensing technology" and variable resistor with a high resistance when unloaded [2]. Ease of calibration was also a big determining factor of this particular sensor, considering it can be done with as little as two to three known loads [2].

$$F_s = k\Delta x,\ \Delta x = \frac{Fs}{k} \quad (1)$$

where k is the spring constant, F_s is the force applied to the spring (determined by the FSR in a voltage divider), and Δx is the change in distance once force is applied. Given the guidance provided in the datasheet, integrating this sensor into a circuit was simple enough. Figure 1 helped in employing an adequate circuit on our motherboard that would have a reasonable resistance value for the voltage divider. With this setup, there was an output resistance range of 1-4.6 kΩ. This datasheet also provided guidance for how to best utilize and calibrate the FSR to constrain the output range to an adequate and measurable range for a data logger [1]. Pictured on the right of Figure 1 is the Voltage vs. Force (grams) calibration curve provided by the datasheet, which was ultimately recreated with a RM value of 10k, an adequate amount for our Teensy measurements and range.

Figure 1. Output Characteristics for an FSR 400 with relationship for voltage (given resistor value in a voltage divider) and Force (g). Schematic for a typical op-amp and voltage divider for the FSR. [1]



Second, the project aimed to evaluate the efficacy of an ultrasonic distance sensor which would serve to get a more direct relationship between voltage and distance. Having a panel for this sensor to reflect waves against is also a major feature of the design. For this particular case, the AUV was outfitted with an HC-SR04. This sensor was lauded for a ranging accuracy reaching 3 mm, and a "2 cm - 400 cm non-contact measurement function" provided it's triggered with a minimum of "10us high level signal" [3].

1 Authors are associated with the Department of Engineering, Harvey Mudd College, Claremont, CA 91711

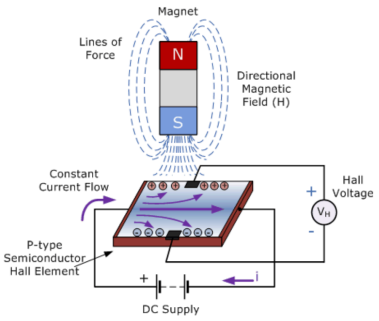
The basic concept behind this sensor is to send a packet of high frequency signals and receive them back when interacting with a close object. Once that occurs, according to the datasheet, distance is evaluated within the sensor which takes into account the speed of sound [3]. The datasheet provides the following equation:

$$\text{Test distance} = (\text{high level time} \times (340\text{m/s}) / 2 \quad (2)$$

where Test distance is the desired result/voltage reading, high level time is a measure of how long the sensor receives a packet, and 340 m/s accounts for the speed of sound. The concept behind this particular sensor is similar to a radar in that it emits an unnatural (and undetectable to the human ear) frequency and waits for that signal to be reflected back, using the wait time to find the distance [4]. Considering some proficiency with a modified 555 timer (elaborated in the Sensor Design section) and other relatively simple setups to adapt this sensor to the Teensy, it serves as a worthy addition. Calculated direct distance (2) and a pre-specified distance in some trials would also be worthy to compare, especially with the regulation of a trigger pulse input. Unfortunately, the sensor was not protected enough considering it likely fried due to the presence of moisture.

The third sensor selected for this project was the AH9246-P-B, a hall effect sensor which would be stationed inside an anemometer frame (allowing for rotation). A hall effect sensor functions as an "ultra high sensitivity Hall-effect switch with internal pull-up resistor on the output" whose output changes with magnetic flux density [5]. Essentially, this functions as a magnetic sensor capable of determining strength and direction of a field. Hall effect sensors are triggered by an external magnetic field and output a certain voltage, which can be adapted into Teensy range and processed appropriately [6]. Figure 2 demonstrates the way a hall effect sensor interacts with a magnet and responds accordingly.

Figure 2. Principles of a Hall Effect Sensor [6]



It's highly applicable in many locations but poses great importance in this context because a moving magnet could trigger this sensor and have an output for the number of rotations in a given time span. This can naturally be adjusted to find the wind speed per the E80 Teaching Team's instructions as they provide a procedure for how wind can rotate a magnet [7]. This sensor was chosen for its ease of use, its supported infrastructure in the overall E80 environment, and to diagnose the ambient conditions to better support the results for distance measurements.

II. Sensor Design

A. Force Sensing Resistor Circuit

Once again, the FSR 400 was one of the primary sensors/resistors being evaluated in this report. A constant 5V supply is necessary for all protoboard circuitry for ease of understanding and troubleshooting. A force-sensing resistor naturally outputs resistance in relation to the force provided, but logging and data analysis is primarily done through the raw voltage exerted by the sensor. As such, a 10kΩ resistor serves as the other half of the voltage divider, a resistor that could get us close to the voltage range needed for proper logging. However, with this setup, the voltage divider voltage was from 3.45 V at 10 N to 4.53 V at 1 N. These obeyed the voltage divider equation:

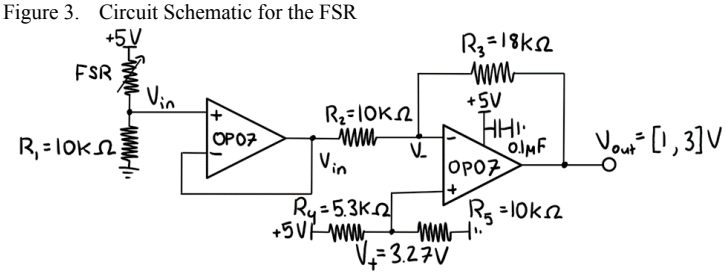
$$V_{out} = \frac{R_g}{R_g + R_{p1}} V_{in} \quad (3)$$

where V_{out} in this equation is V_{in} in Figure 3, R_g is the 10kΩ resistor, and R_{p1} is the FSR represented as a variable resistor. The rest of the schematic depicted in Figure 3 is simply scaling and offset operational amplifiers (op-amps) needed to get the output voltage range within Teensy range. To have the output voltage properly read (0.3-3.3V Teensy range), that offset amplifier in Figure 3 is necessary. It would obey the following equation:

$$V_{out} = \left(1 + \frac{R_f}{R_{n1}}\right) V_+ \left(\frac{R_g}{R_{p1} + R_g}\right) - \frac{R_f}{R_{n1}} V_{in} \quad (4)$$

where V_{OUT} is the voltage reading seen by the Teensy, R_f is R_3 in the figure needed for a 1.8 gain. R_{n1} is the 10kΩ resistor in between the op-amps, and finally, R_g and R_{p1} coincide with the 10 and 5.3 kΩ resistors under the second op-amp. These aim to get a 3.27 V offset and results in a V_{OUT} with a range of 1-3V, appropriate for the necessary range a Teensy can verifiably read. The expected range of input values for this circuit are forces from 0-10 N though this range will be limited to 0-3N experimentally. This translates to a variable resistance informed by the following calibration. No significant software modification was made for this sensor. The ADC sampler (outputting a raw voltage reading from a Teensy pin) was primarily used with a modified sampling period of 69 ms or 14.4 Hz sampling rate to better visualize FSR resistance changes.

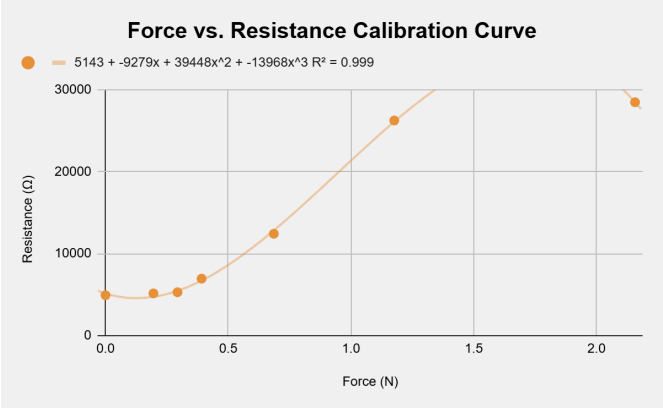
Figure 3. Circuit Schematic for the FSR



B. FSR Calibration and Setup

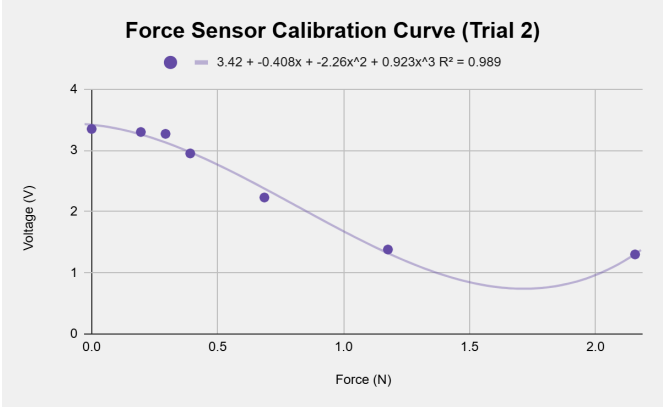
The force-sensitive resistor was calibrated by applying known forces between 0-10 N and measuring the output voltage through the circuit. However, for the sake of accuracy the Resistance and Voltage relationships were recreated and validated using calibration curves. This is due to the complexity of the sensor when relating both variables, so focus generally shifted to detecting any contact.

Figure 4. Force vs. Resistance Calibration Curve



A third degree polynomial for this calibration immensely well and followed the expected behavior of the FSR elaborated in the FSR 400 datasheet [1].

Figure 5. Force vs. Voltage Calibration Curve



The next step was to properly calibrate the known force application to a voltage reading using our developed circuit in Figure 3. The relationship depicted in Figure 5 was found to be the most appropriate one, having an R^2=0.989 and properly reading the voltage output of our circuit as put through an oscilloscope as force is increased. The reason for the discrepancy in these calibrations when compared to the datasheet is the little detail in the equation provided in the datasheet:

$$V_{OUT} = \frac{RM \times V+}{(RM+RFSR)} \quad (5)$$

The datasheet mentions what would happen if R_M and R_{FSR} would swap, resulting in an output swing or sign direction flip [1]. This would eventually be adjusted to find the proper distance relationship between force and distance using Hooke's Law and mechanical testing. Testing the sensor on a properly developed mechanical frame (week of April 4th) allowed the derivation of distance when related to voltage. Figure 6 shows exactly that, a procedure carried out over multiple trials to develop an appropriate uncertainty and derive a distance measurement in relationship to Force and Voltage.

Figure 6. Force, Voltage and Distance Related using FSR Circuit (with uncertainty)

Given Force	Voltage	Distance	Unc.
0	3.58	-0.10432	0.126864293
0.195219	3.12778956	-0.01206907	0.116255821
0.391419	2.721918832	0.070728558	0.101189027
0.685719	2.204488916	0.176284261	0.078102816
1.176219	1.585775529	0.302501792	0.054546854
2.157219	1.262110524	0.368529453	0.050326757

The last required calibration necessary for this application of the FSR 400 is to properly assess the spring contained in the mechanical assembly to relate the sensed force (from the FSR) to a distance, able to be compared against the ultrasonic distance sensor.

Figure 7. Spring Calibration Curve

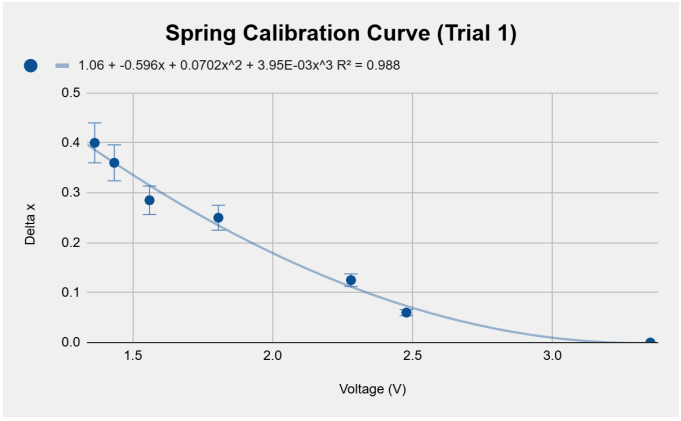


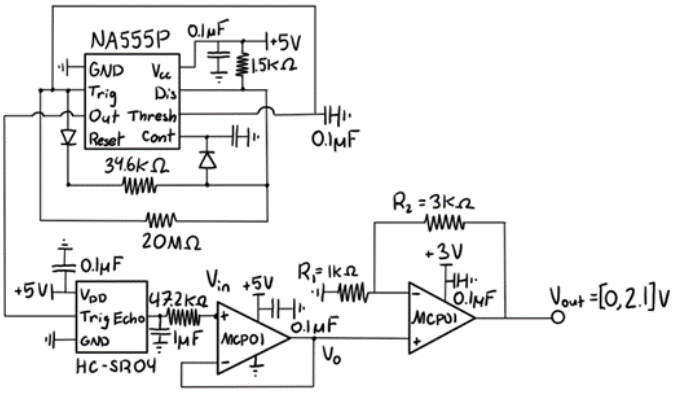
Figure 7 shows the way in which the spring constant was determined. This is a necessary component of Hooke's Law and was instrumental in validating the theory behind the way this particular sensor would work. A strong R^2 value and the uncertainties suggest the relationship between Voltage readings from the FSR circuit and distance here are confident.

C. Ultrasonic Distance Sensor Circuit

The acoustic ToF sensor was calibrated by measuring the average peak voltage over 10 trials in 10cm increments between 0.1-1m. The circuitry behind this sensor was undeniably the most complex. First, a 555 timer (NA555P) was utilized with a modified pulse width (astable configuration) to go into the Trig input of the HC-SR04 ultrasonic distance sensor. The diodes are responsible for the astable configuration. This configuration is shown in Figure 8 and essentially controls the timing of the output pulses and charges and discharges capacitors (to get a steady reliable input). This would input high frequency square waves into the trig pins at a specified pulse width every 345 ms (adjusted with the feedback resistors in the 555 timer). This would trigger the

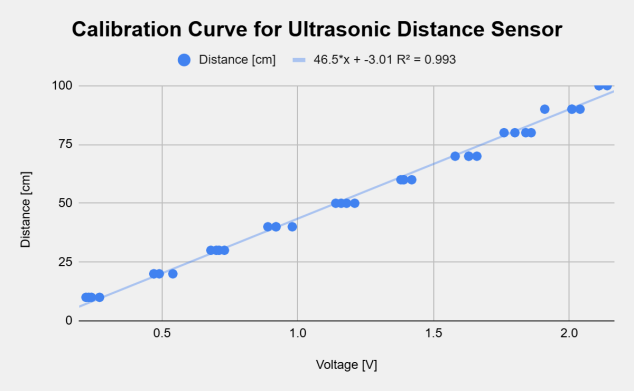
sensor which would send packets and output a rectangular wave with a variable duty cycle from the echo pin of the HC-SR04 [8]. As waves are sent out, they reflect on an object and are picked up again, providing an output that can be appropriately scaled to Teensy range. Originally, the output of the HC-SR04 was 0-500 mV for a range of 0-100 cm. But, with a non-inverting amplifier (as described before) with a gain of 4, the output can be changed. Thus, the output of the circuit can be expected to range from 0 to 2.1 V, corresponding to about 1 meter in distance.

Figure 8. Circuit Schematic for the Ultrasonic Distance Sensor



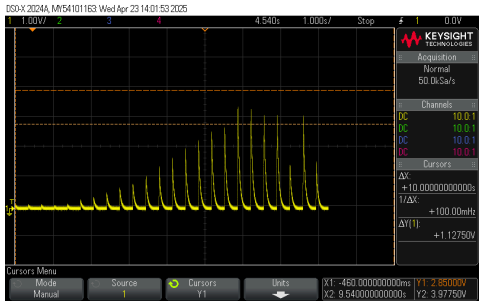
The calibration was relatively simple and played out as explained before. A relationship was established and fit linearly with an R^2 0f 0.993. This range was also appropriate for deployment at Baby Beach, Dana Point.

Figure 9. Ultrasonic Distance Sensor Calibration Curve



To further confirm the validity of the circuit design, a dry test was conducted after putting the circuit into the protoboard interface of the AUV. Figure 10 shows how the Echo pin outputs and scales given a variable distance. The oscilloscope trace also helped to understand the range provided by this setup, which only reached about 1.2 m, rather than the 4 m specified in the datasheet [4].

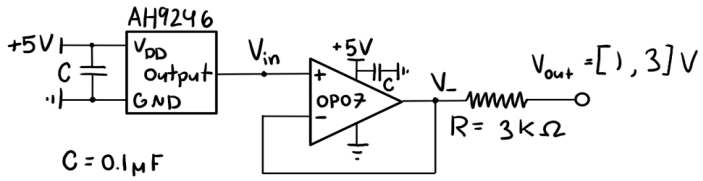
Figure 10. Oscilloscope Test for Output Range in Dry Test with Ultrasonic Distance Sensor



D. Hall Effect Sensor (as part of the Anemometer)

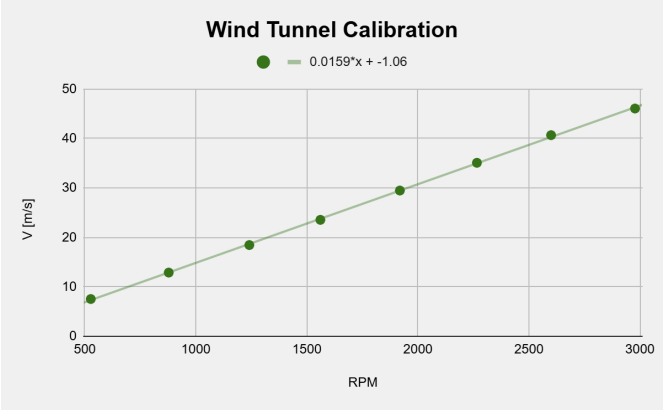
Finally, the sensor design for the Hall Effect Sensor was primarily constructed to have the sensor in an elevated frame on the robot. As such, the sensor circuitry itself was pretty simple considering the sensor already has a decent output that can be read by a Teensy. The AH9246 served as an appropriate sensor because only one op-amp (the OP07 pictured in Figure 11) was necessary to have the output voltage range be from 1-3 V. Considering the sensor is supposed to determine the rotations of the anemometer mechanical frame, only an analog output was needed to convert into rotations over time. However, what this sensor lacked in circuit complexity, it made up for in the necessary calibration steps and software modifications.

Figure 11. Circuit Schematic for the AH9246 Hall Effect Sensor



The anemometer was calibrated using a wind tunnel by measuring the rotations per second (RPS) for a given input RPM. Because of this, the wind tunnel being used for calibration needed to be analyzed (Figure 12). This was needed to understand the correlation between its fan RPM (displayed as part of the wind tunnel) and the wind speed in m/s. This allows the further calibration of the anemometer frame (constructed using guidance for the E80 teaching team) to find the RPS of the anemometer and convert that into wind speed. Using a given RPM of the wind tunnel, it can be determined how RPS changes with different wind speeds.

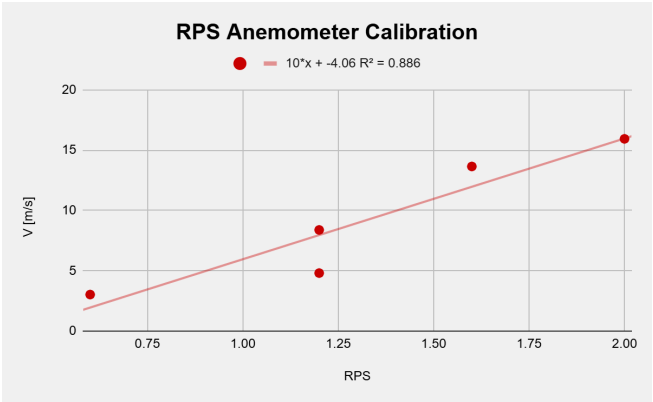
Figure 12. Wind Tunnel Calibration Curve



Knowing the relationship between wind tunnel RPM and wind speed was integral. Now, wind speed can be varied independently and RPS of the anemometer could be determined experimentally. This was done with a myriad of software modifications to the base loop that ran the ADC sampler. The hall effect sensor (triggered every time the magnet passed by) would pass a reading to the A0 pin on the Teensy. The software can read this pin, and throughout every loop, track when voltage drops significantly. This allows the code to incrementally read the rotations and track the time span. Due to the software modifications and continuous reading, a new library named *AnemSampler.m* (see Appendix 2) had to be created to find RPS as a function of time, obeying the 69 ms sampling rate.

Figure 13 demonstrates how tracking the RPS of the anemometer was connected to the velocity the wind tunnel was set at. With this calibration and software, the project could move forward and count rotations over time in order to convert to wind speed. This sensor serves to test ambient conditions and supplement the other sensors to explain any strange behavior.

Figure 13. Anemometer Mechanical Assembly RPS Calibration Curve



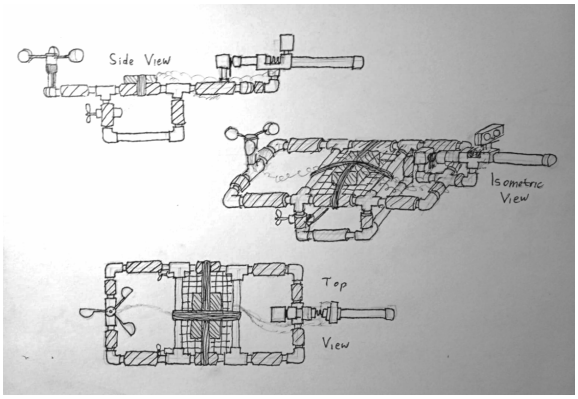
III. Robot Design

A. *Overall Mechanical Frame*

The mechanical frame proved to be an interesting challenge in the development of this AUV. Motors had to be underwater to propel the AUV forward, though they also had to respond to the outputs of the FSR 400 and HC-SR04. The anemometer had to be properly elevated to receive significant ambient conditions and wind speed. In hindsight, more elevation would have led to more significant results from the hall effect sensor. Besides these details, the overall frame had to have a large surface area considering the extrusion made for the FSR 400 (pictured in Figure 14).

A large surface area would prevent tipping from the bot, though it would also add weight. Due to the added weight on the top of the AUV, appropriate considerations had to be taken for center of mass (COM) and center of buoyancy/pressure (COB). COM had to be below the center of buoyancy so the AUV would be hard to flip over and would have self-righting behavior. Pool noodles were added to the frame to assist in that venture and the box housing the Teensy, motherboard and protoboard was kept above water for safety reasons. Any further consideration was placed on particular sensor needs, whether that be the necessary mechanical finesse for the FSR 400, or the frame enabling the hall effect sensor to count rotations.

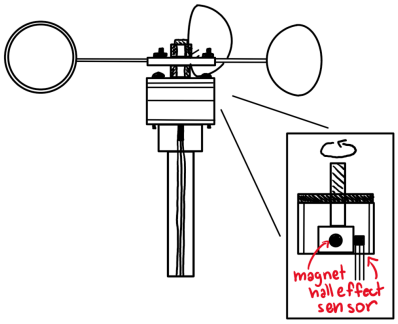
Figure 14. Overall Mechanical Assembly, Multiple Views



B. *Anemometer Mechanical Design*

The mechanical frame of the anemometer has been alluded to many times in this report. The anemometer houses a magnet which will "be used to mark the beginning of one revolution. When the wind pushes the cups around, the magnet will pass by the hall effect sensor" [7]. Essentially, the cups will be pushed as air flows by, rotating the magnet and triggering the sensor. Figure 15 depicts the interior of the anemometer mechanical assembly, demonstrating how the hall effect sensor would interact with the magnet being moved around by rotations of the cups.

Figure 15. Anemometer Mechanical Assembly demonstrating ingrained hall effect sensor and accompanying magnet

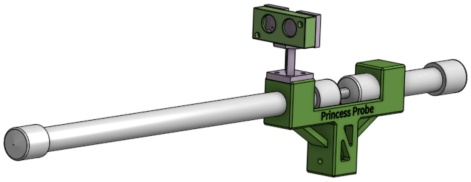


From there, leads had to be soldered on the hall effect sensor to provide power, ground, and read the output of the magnet for every revolution. In terms of balance, this sensor did significantly tilt the axis of the overall mechanical frame. This explains the second mechanical construction on the AUV, housing the other two sensors evaluated in this project.

C. *FSR 400 & HC-SR04 Mechanical Design*

Figure 16 demonstrates a CAD model of the device made to house a spring, 14.5" extrusion that would change in length when force was applied, and the frame where the ultrasonic sensor would lie. This would be mounted on to the AUV and balance out the mass distribution overall. The primary function of this extrusion is to practically observe Hooke's Law. When this extrusion is provided with force, the proximity to an object can be evaluated given a known spring constant. Granted, this setup does require contact with the object, which is the reason the extrusion measures 16 inches, somewhere within the range of the ultrasonic sensor. The frame for the ultrasonic sensor is elevated to prevent any possible extrusion and for the ability to have a panel to reflect against.

Figure 16. Ultrasonic Distance Sensor Mechanical Frame and Spring+Arm Assembly Triggering FSR



Besides these features, the design primarily considered ease of transportation, proper distribution of mass, and reasonable leads to connect each sensor to the box housed in the middle of the AUV.

IV. Deployment Details

A. *Procedure*

Since it rained at Dana Point during deployment, the HC-SR04 fried since it was not waterproof. This was compensated in the mechanical design though the effect of rain on the performance was not adequately considered. It caused a short in our AUV which also affected the AH9246 hall effect sensor. Fortunately, a replacement was found. However, adjustments had to be made.

Deployment consisted of inserting an SD card into the motherboard, flicking a switch on the motherboard to start the systems, then closing off the circuitry into a harnessed box. During deployment, wind speed would be monitored in the Serial Monitor through Arduino. Afterwards, the raw voltages would be extracted from each trial to get an idea of the performance of each sensor. This data in the SD card would be run through MATLAB code (see Appendix 3) and moderately processed. Considering the simplicity of the output of each sensor, any processing and analysis would be through referencing the previous calibrations and taking into account those errors. Wind speed from the anemometer would be primarily analyzed through the output of the Serial Monitor.

The software was edited to run in a continuous line until motors felt a high enough force from the FSR 400 (See Appendix 1). This would indicate close proximity and the motors would consequently calm down so the steps could be repeated. Figure 17 shows deployment in action as the AUV was pressed against a team member so distance from the team member could be determined.

Figure 17. Image of Dana Deployment Procedure near the shore



The anemometer functioned well enough and was occasionally manually spun as a trial began to ensure its operation. Data acquisition consisted of inserting an SD card into the motherboard, flicking the switch and harnessing the shut box onto the AUV for safety. The AUV would then be retrieved after deployment and the SD card would be put into a MATLAB file named *logreader.m* which would interpret the Teensy output (See Appendix 3).

B. *Safety and Environmental Concern*

To reduce the environmental impact of the project, the AUV was deployed at the shore to minimize the probability a piece of the AUV would be lost. This nearly happened before in a previous unsuccessful deployment at a lake. This would prevent pollution and any significant impact on sea life. Furthermore, the performance of the circuitry onboard the AUV was assessed before and after deployment.

V. Results

Ultimately, data for the ultrasonic distance sensor could not be acquired and read as completely constant despite attempts at triggering the sensor with a panel in close proximity. The anemometer responded as expected with analog readings and variable outputs coinciding with the way the anemometer was turned. The results also obeyed the range specified in the circuit schematic. However, some discrepancies happened in the results of

the FSR 400, which will be elaborated further. Regardless, the key results of this project are the assessment of two trials, each with a different motor pulse width modulation (PWM). The higher the PWM, the faster the motors will run. This gives the ability to compare each trial and how the FSR responds accordingly, allowing us how the distance can be determined.

Figure 18. Dana Deployment 1 Results at 120 PWM

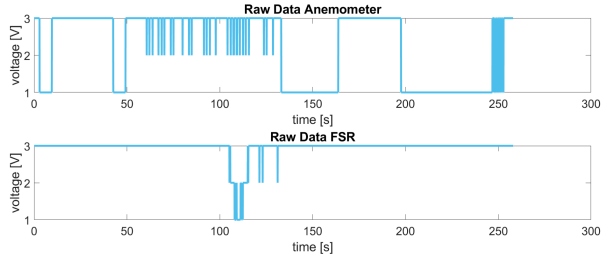
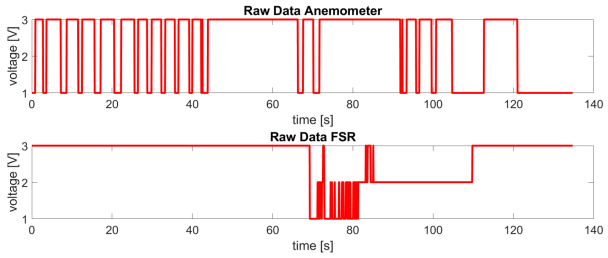


Figure 18 shows the raw data taken from Deployment 1, running at 120 PWM. The FSR did not behave as expected, being pretty digital rather than showing the slow change in force magnitude during deployment. This also didn't match dry tests processed with MATLAB code and the calibration curve. Those typically demonstrated force being variable over time.

This discrepancy from modeled data (as done in dry tests) can likely be from a faulty sampling period. The FSR does have intermediate values present so it's likely that aliasing occurred. This deployment is particularly interesting because when compared to video, the response in the FSR is valid, though the incremental and slow change is not captured. If the software provided has a sampling rate of 14.4 Hz, that means the relative speed of the motors would outpace the changes in force captured.

Figure 19. Dana Deployment 2 Results at 290 PWM



The MATLAB processing code for the anemometer simply displays the output of the hall effect sensor. This appears to be functioning just fine. The real results from the anemometer comes from the software which continuously finds wind speed. This software, as observed on the terminal, typically found a range of wind speeds from 2 m/s to 6.21 m/s. These coincide with the ambient conditions observed on that day reporting an average wind speed of 14 mph or 6.25 m/s. Discrepancy likely occurred due to the low elevation of the anemometer.

VI. Conclusion

This work gave insight into the creative applications of sensors not normally used to take certain kinds of measurements. It also highlighted the different needs each sensor requires, whether that be mechanical design, software modification, or intensive circuit design.

Future work would likely involve a proper integration of the ultrasonic distance sensor such that the output is an analog voltage proportional to distance and the sensor is further waterproofed. A control loop software modification (similar to the anemometer) can also be taken off to control distance from an object using motor PWM.

If another chance of deployment was possible, a new ultrasonic distance sensor could be acquired, newly calibrated, and a few software modifications could be made to the sampling frequency to prevent any aliasing or difficulty of post-processing. Generally, it may also be valuable to invest in more established methods of measuring distance through sensors.

Acknowledgements

Special thanks to Prof Joshua Brake, Prof. Dre Helmns, Lynn Kim, and Xavier Walter. They each provided valuable guidance that furthered the exploration of this study into novel areas. Prof. Lynn Kim excelled in guiding us through the complex setup of the ultrasonic distance sensor and Xavier Walter helped in terms of software and the general approach of the research. Furthermore, thank you to the proctors Jasper Cox, Kavi Dey, Naomi Horiguchi, Kayla Long, and Katrina Nguyen. Troubleshooting would have likely gone on longer if not for their insight and expertise.

References

[1] I. E. Inc, "FSR 400," www.interlinkelectronics.com. https://www.interlinkelectronics.com/fsr-400
[2] Tekscan, "How Does a Force Sensing Resistor (FSR) Work?," Tekscan, Jul. 03, 2019. https://www.tekscan.com/blog/flexiforce/how-does-force-sensing-resistor-fsr-work
[3] Digi-Key Electronics, "HC-SR04 Datasheet by SparkFun Electronics | Digi-Key Electronics," Digikey.com, 2024. https://www.digikey.com/htmldatasheets/production/1979760/0/0/1/hc-sr04.html
[4] J. S. Cook, "Ultrasonic Sensors: How They Work (and How to Use Them with Arduino)," Arrow.com, Apr. 04, 2018. https://www.arrow.com/en/research-and-events/articles/ultrasonic-sensors-how-they-work-and-how-to-use-them-with-arduino
[5] Diodes Incorporated, "Hall Effect Latches and Switches," Diodes Incorporated, Jan. 31, 2022. https://www.diodes.com/part/view/AH9246 (accessed May 06, 2025).
[6] ElectronicsTutorials, "Hall Effect Sensor and How Magnets Make It Works," Basic Electronics Tutorials, Feb. 09, 2018. https://www.electronics-tutorials.ws/electromagnetism/hall-effect.html
[7] E80 Teaching Team. "Reference Anemometer Procedure." Internet: https://docs.google.com/document/d/1rY2bEjX3RP1EuHBjaoB50aiPz61I9q4W86KAzeNQdSo/edit?tab=t.0 [Mar. 27, 2025].
[8] A. Singh, "Ultrasonic Sensor Without Arduino using 555 timer» Hackatronic," Hackatronic, Jan. 30, 2021. https://hackatronic.com/ultrasonic-sensor-without-arduino-using-555-timer/ (accessed May 06, 2025)

Code Appendix 1: `DanaDrive.ino` Partial Function for Motor PWM

```
currentTime=millis();
if (forceSensor <= 500) {
  motor_driver.drive(290,290,0);
 }else {
  motor_driver.drive(-150,0,0);
 }
```

Code Appendix 2: `DanaDrive.ino` Partial Function for Anemometer Wind Speed Calculation

```
  if  (  currentTime-anem_sampler.lastExecutionTime  >
LOOP_PERIOD) {
   anem_sampler.lastExecutionTime = currentTime;
   float timeDifference = currentTime - endTime;

   if (timeDifference > sampleTime) {
   // RPS/
   RPS = (revolution / sampleTime) * 1000;
   anem_sampler.updateState(RPS);

   endTime = currentTime;
   revolution = 0;
 }
…
void ISR(){
 revolution += 1;
}

double calculateAverage(double sampleSet[], int N) {
   double sum = 0.0;

   for (int i = 0; i < N; i++) {
      sum += sampleSet[i];
   }

   return sum / N;
}
```

Code Appendix 3: `logreader.m` Function necessary to interpret Teensy Output from each ADC terminal

```
% plottinglab1.m
% plotting code written by Lily Anfang, lanfang@g.hmc.edu,
Jan 26 2025
%


%clf;
clear
clf
filenum = '125'; % file number for the data you want to read
infofile = strcat('INF', filenum, '.TXT');
datafile = strcat('LOG', filenum, '.BIN');

%% map from datatype to length in bytes
dataSizes.('float') = 4;
dataSizes.('ulong') = 4;
dataSizes.('int') = 4;
dataSizes.('int32') = 4;
dataSizes.('uint8') = 1;
dataSizes.('uint16') = 2;
dataSizes.('char') = 1;
dataSizes.('bool') = 1;

%% read from info file to get log file structure
fileID = fopen(infofile);
items = textscan(fileID,'%s','Delimiter',',','EndOfLine','\r\n');
fclose(fileID);
[ncols,~] = size(items{1});
ncols = ncols/2;
varNames = items{1}(1:ncols)';
varTypes = items{1}(ncols+1:end)';
varLengths = zeros(size(varTypes));
colLength = 256;
for i = 1:numel(varTypes)
   varLengths(i) = dataSizes.(varTypes{i});
end
R = cell(1,numel(varNames));

%% read column-by-column from datafile
fid = fopen(datafile,'rb');
for i=1:numel(varTypes)
   %# seek to the first field of the first record
   fseek(fid, sum(varLengths(1:i-1)), 'bof');

   %# % read column with specified format, skipping required
number of bytes
   R{i} = fread(fid, Inf, ['*' varTypes{i}],
colLength-varLengths(i));
   eval(strcat(varNames{i},'=','R{',num2str(i),'};'));
end
fclose(fid);
accelX = accelX/1000;
accelY = accelY/1000;

%% Process your data here

%teensy sampling period is about 4.75us and teensy unit is
about 0.0032V from lab 2
n = 0:1:(length(A00)-1);
t = 0.069 .*n;
subplot(3,1,1)
```

```
plot(t, 0.0032.*A00)
xlabel("time [s]")
ylabel("voltage [V]")
title("raw data anemometer")

subplot(3,1,2)
plot(t, 0.0032.*A01)
xlabel("time [s]")
ylabel("voltage [V]")
title("raw data FSR")

%subplot(3,1,3)
%plot(t, 0.0032.*A03)
%xlabel("time [s]")
%ylabel("voltage [V]")
%title("raw data acoustic ToF")

%Distance = 0.0214*V + 0.0728
%Period = 0.345s
%want sampling period to be 69ms

l=(length(A03)-mod(length(A03),5))/5;
high=zeros(1,l);
for x = 1:1:l
   points = (5*x-4):1:5*x;
   high(x) = 0.0032.*max(A03(points));
end
high = 46.5*high-3.01;
tnew = 0.345*(1:1:length(high));
%subplot(4,1,4)
%Tplot(tnew, high)
%xlabel("time [s]")
%ylabel("distance [cm]")
%title("calibrated distance from boat (ToF data)")
```